

Product Design & Architecture (PDA): “AuroraLog” – Application Logging Library

1. Document Info

Product name (working): AuroraLog (placeholder)

Owner:

Status: Draft v0.1

Last updated: 2025-12-09

2. Background & Problem Statement

Most of our internal projects currently use console.log, ad-hoc custom loggers, or framework-specific logging that:

- Produces inconsistent log formats across services.
- Makes it hard to filter or search logs (no structured metadata).
- Couples logging behavior to environment (dev vs prod) in fragile ways.
- Lacks central control over log levels, sinks (file, HTTP, cloud), and performance.

For production-grade systems and open-source consumers, we need a standardized, extensible logging library that:

- Is easy to adopt (console.log-level friction).
- Provides structured logs and consistent semantics.
- Can be used both in development and production environments.
- Is language-runtime friendly (JS/TS, Node.js + browser).

3. Goals & Non-Goals

Goals:

1. Replace console.log.
2. Support standard log levels.
3. Provide structured logging JSON metadata.
4. Support multiple transports (console, file, HTTP, memory).
5. Provide pluggable architecture (custom transports, custom formatters).
6. Production-ready performance + reliability.
7. Strong developer experience.

Non-Goals:

- Full distributed tracing system.
- Observability platform.
- Multi-language implementation (v1 JS/TS only).
- Log indexing engine.

4. Target Environment & Assumptions

- Primary runtime: Node.js (LTS)
- Secondary runtime: Browser
- Consumers: internal backend, frontend, external OSS users.

Assumptions:

- Apps call init function early.
- Node apps may access filesystem.
- Network failures possible → graceful degradation.

5. Use Cases & User Stories

Core Use Cases:

1. Replace console.log with structured logger.
2. Filter logs by level per environment.
3. Attach request context (requestId, userId).
4. Send logs to central HTTP endpoint.
5. Write logs to rotating files.
6. Capture unhandled errors + rejections.
7. Testing: capture logs in-memory.

Secondary:

- Per-module loggers.
- Dynamic log-level reload.
- Redaction.
- Telemetry integration.

6. Functional Requirements (Features)

6.1 Core Logging Features

- Log levels: TRACE, DEBUG, INFO, WARN, ERROR, FATAL.
- Logging API: `logger.info(message, meta?)`
- Structured payload with timestamp, level, message, metadata.
- Context propagation: `logger.withContext({requestId})`

6.2 Transports

Console:

- Dev: pretty + color
- Prod: JSON

File:

- Append
- Rotation (size/time)
- Compression optional

HTTP:

- Send immediately or batch
- Retry + backoff
- Dropping policy under pressure

Memory:

- For tests: capture + retrieve

Custom transports:

- Standard interface

6.3 Formatting

- JSON formatter
- Pretty formatter
- Custom formatters
- Message templating

- Redaction rules

6.4 Configuration

- initLogger(config)
- Config from object + env vars
- Presets: dev/test/prod

6.5 Error Handling

- Serialization errors
- Transport failures
- Global error integration (uncaughtException, unhandledRejection)

6.6 Integration

- Optional console.patch()
- Async context
- Framework adapters (future)

6.7 DX & Tooling

- TS-first API
- Tree-shakeable for browser
- Excellent docs

7. Non-Functional Requirements

- Performance: avoid overhead when levels disabled, batching.
- Reliability: logging must not crash app.
- Security: redaction, TLS.
- Portability.
- Maintainability: modular.
- Testability: memory transport for assertions.

8. High-Level Architecture

Core:

Logger → LogDispatcher → {Transports}

Formatters

Config

Context module

9. API Sketch

```
createLogger({ level, transports, defaultContext })
```

```
logger.info("message", {meta})
```

```
reqLogger = logger.withContext({ requestId })
```

10. Operational Considerations

- Monitoring buffer state.
- Changing log levels without redeploy.
- SemVer compatibility.

11. Future Enhancements

- OpenTelemetry integration.
- CLI for log viewing.
- Cloud logging adapters.
- Language ports.

12. Open Questions

- Global singleton or explicit creation?
- Browser context propagation?
- Default redaction list?
- Config merging precedence?
- Node vs browser build strategy?